Universidad Tecnológica Nacional · FRC

Gestión de Configuración del Software (SCM)

Cátedra de Ingeniería y Calidad de Software · Docentes: Judith Meles, Laura Covaro · *"Más allá del Commit, Update"*

Definición Formal (ANSI/IEEE 828, 1990)

Disciplina que aplica dirección y monitoreo técnico y administrativo a: **identificar y documentar** las características funcionales y físicas de los ítems de configuración, **controlar los cambios** en tales características, **registrar y reportar** el estado de implementación de los cambios y **verificar** la correspondencia con los requerimientos acordados.

1. Contexto del Software y Necesidad de SCM

El Software en Contexto

El software no es solo código fuente; es un conjunto articulado de **programas, procedimientos, reglas, documentación y datos**. Se desarrolla a través de un **Proceso** adaptado, impulsado por **Personas**, automatizado con **Herramientas**, gestionado en un **Proyecto** para obtener como resultado un **Producto** íntegro.

El Software: Un "Blanco Móvil"

Los cambios en el software son inevitables y provienen de múltiples orígenes:

- Cambios en las reglas de negocio y nuevos requerimientos.
- Soporte y compatibilidad con productos asociados.
- Reorganización de prioridades corporativas o de mercado.
- Ajustes presupuestarios, defectos y oportunidades de mejora.

2. SCM como Disciplina de Soporte ("Actividad Paraguas")

SCM es una actividad transversal que acompaña al software durante todo su ciclo de vida. Coexiste y colabora con otras disciplinas clave de soporte en ingeniería de software:

Disciplina de Soporte	Objetivo Principal	Relación con SCM
SCM (Gestión de Configuración)	Garantizar la integridad, trazabilidad y control de evolución de los artefactos.	Provee los artefactos controlados y líneas base que serán probados y asegurados.

Control de Calidad (QC)	Verificar la calidad del producto y del proceso construido.	Evalúa artefactos estables identificados bajo una línea base específica.
Pruebas de Software (Testing)	Detección sistemática de fallas y discrepancias respecto a requisitos.	Requiere certeza exacta de qué versión y fuente compone el ejecutable probado.
Aseguramiento de Calidad (SQA)	Auditar el cumplimiento de procesos, estándares y metodologías de trabajo.	Fiscaliza que los procedimientos de control de cambios y auditorías se ejecuten.

Riesgos Críticos de No Implementar SCM

Pérdida de componentes, pérdida de cambios (sobrescrituras accidentales), desfase de sincronía fuente-objeto-ejecutable, regresión de fallas ya resueltas, doble mantenimiento divergente, superposición destructiva y despliegue de cambios no validados.

3. Conceptos Clave de la Gestión de Configuración

Para establecer orden y control sistemático sobre la evolución de un sistema, SCM se estructura a partir de conceptos fundamentales:

Ítem de Configuración de Software (SCI / IC)

Cualquier artefacto que forma parte del producto o del proyecto, sujeto a cambios, que necesita compartirse entre los miembros del equipo y del cual se requiere conocer con exactitud su estado, origen y evolución.

Configuración del Software

Es el conjunto coordinado de ítems de configuración con sus versiones específicas correspondientes en un instante determinado del tiempo (representación instantánea o *snapshot*).

Clasificación de Ítems de Configuración (SCI)

Ámbito / Categoría	Artefactos Comunes Incluidos

De Producto	Especificación de Requerimientos (ERS/SRS), Documento de Arquitectura, Código fuente, Librerías, Gráficos e íconos, Manuales de Usuario, Scripts de instalación/despliegue.
De Proyecto	Plan de Proyecto, Plan de Gestión de Configuración (CMP), Cronograma general, Matriz de riesgos, Visión, Guía de estilo IHM, Estándares de codificación.
De Iteración / Soporte	Planes de Iteración / Sprint, Casos de Prueba, Reportes de Defectos, Propuestas de Cambio formalizadas, Listas de control de entrega y Formularios de aceptación.

4. Repositorios, Versiones, Ramas y Variantes

El **Repositorio** es el contenedor seguro que almacena los SCI, preserva su historial completo con atributos y relaciones, y permite evaluar el impacto de los cambios propuestos (compuesto por una o varias bases de datos con operaciones de *Check-out*, *Check-in* y actualización).

Repositorios Centralizados (ej. SVN)

Un único servidor central contiene todos los archivos y versiones. Facilita un control y gobierno centralizado por los administradores; su vulnerabilidad es que si falla el servidor central se detiene la sincronización.

Repositorios Distribuidos (ej. Git)

Cada cliente mantiene una copia exacta y completa del repositorio e historial. Si el servidor falla, se restaura desde cualquier cliente. Habilita flujos de trabajo avanzados y desarrollo desconectado.

Versión vs. Variante

Versión: Estado particular de un IC en un instante o contexto determinado (evolución lineal representada como grafo).
Variante: Versión que evoluciona por separado para soportar configuraciones alternativas (ej. multi-plataforma: Windows/Linux, módulos optativos).

Ramas (Branches) y Merge

Permiten bifurcar el flujo principal (*trunk/master*) para experimentar o aislar tareas sin desestabilizar la base. Las ramas deben integrarse eventualmente a la rama principal mediante la operación **Merge** (resolviendo conflictos con herramientas de *diff*) o descartarse.

5. Líneas Base (Baselines)

Definición y Rol en el Ciclo de Vida

Una **Línea Base** es una configuración que ha sido formalmente revisada y consensuada, sirviendo como punto de partida indiscutible para desarrollos posteriores. **Sólo puede modificarse mediante un procedimiento formal de control de cambios.** Permite volver atrás en el tiempo, reproducir con fidelidad exacta el entorno de construcción y dar estabilidad al desarrollo.

Las líneas base se marcan habitualmente mediante *etiquetas (Tags)* en el repositorio (no confundir la etiqueta de línea base con la versión comercial del producto):

1. Planificación

Línea Base del Plan

2. Requisitos

Línea Base de Requisitos

3. Diseño

Línea Base de Diseño

4. Codificación

Línea Base de Fuentes

5. Pruebas

Línea Base de Prueba y Release

6. Actividades Fundamentales de SCM

El marco formal de SCM se articula en cuatro pilares sinérgicos:

1. Identificación de Ítems

Asignación de identificadores unívocos a cada artefacto, definición de reglas de nomenclatura, establecimiento de esquemas de versionado y estructuración interna del repositorio.

2. Control de Cambios

Mecanismo estructurado que evalúa el impacto técnico y comercial, previene cambios anárquicos sobre artefactos en línea base y administra las solicitudes formales de modificación.

El Circuito Formal de Control de Cambios

Etapa	Acción Realizada	Salida / Responsable
1. Petición	Detección de necesidad por defectos, negocio o clientes.	Solicitud de Cambio formal (Change Request / PR).
2. Análisis	Evaluación técnica de impacto en costos, plazos, arquitectura y riesgos.	Informe de impacto y efectos colaterales.
3. Dictamen	Revisión multidisciplinaria por el **Comité de Control de Cambios (CCB)**.	Aprobación, Rechazo o Postergación formal.
4. Ejecución	Generación de Orden de Cambio de Ingeniería; asignación en la cola.	Desarrollo, verificación y nueva línea base.

El Comité de Control de Cambios (CCB / CCC)

Órgano colegiado formado por representantes de todas las áreas involucradas: **Análisis/Negocio, Arquitectura/Diseño, Desarrollo, Aseguramiento de Calidad/Testing y Gestión de Producto**. Es la única entidad facultada para autorizar modificaciones a las líneas base aprobadas.

7. Auditorías de Configuración e Informes de Estado

Aseguran la correspondencia exacta entre los requerimientos contractuales, los documentos de diseño y el producto tangiblemente construido:

Auditoría Funcional de Configuración (FCA)

Evaluación independiente que controla que la funcionalidad real y el desempeño del producto coincidan cabalmente con la especificación de requerimientos (SRS). Apoya la **Validación** ("¿Construimos el producto correcto?").

Auditoría Física de Configuración (PCA)

Verificación técnica de que el software y su documentación están completos, construidos e integrados tal como se declaró en la línea base o release note. Apoya la **Verificación** ("¿Construimos el producto correctamente?").

Registro e Informe de Estado (Configuration Status Accounting)

Mantiene la historia viva del sistema y suele automatizarse con herramientas de integración continua. Permite responder con certeza auditora: *¿En qué estado está cada IC? ¿Qué versión implementa tal requerimiento aprobado? ¿Qué cambios exactos existen entre dos versiones?*

8. Plan de SCM y Enfoque en Entornos Ágiles

¿Qué contiene el Plan de SCM (CMP)?

- Reglas de nombrado y codificación de los ICs.
- Estructura de ramas y herramientas oficiales (Git, CI/CD).
- Roles, responsabilidades y miembros del CCB.
- Flujo formal de aprobación y plantillas de peticiones.
- Criterios de auditoría y políticas de release.

SCM en Ambientes Ágiles (Agile SCM)

- **Servicio al equipo:** SCM asiste al desarrollador en lugar de entorpecer con burocracia rígida.
- **Automatización sin fricción:** Integración y Despliegue Continuo (CI/CD), pruebas automáticas y trazabilidad Lean.
- **Responsabilidad compartida:** Todo el equipo es custodio de la configuración y la calidad de la rama principal.
- **Adaptación al cambio:** En lugar de resistir el cambio, se incorporan prácticas de *pull requests* y revisiones por pares continuas.

Referencias Bibliográficas Clave

• **ANSI/IEEE Std 828-1990 / 2012:** Standard for Configuration Management in Systems and Software Engineering.
• **Bersoff, E. H. (1984):** *"Elements of Software Configuration Management"*, IEEE Trans. Softw. Eng., 10(1), 79-87.
• **SPMN:** *Little Book of Configuration Management* · **Berczuk & Appleton:** *Software Configuration Management Patterns: Effective Teamwork, Practical Integration*.